Тема 8

- статична функция - обвързва се с една компилационна единица (1 .cpp файл), еквивалент на функция в анонимен namespace (namespace { void f() {}})
  ↳ все едно е static

- локална статична променлива - стои при глобалните променливи в паметта; инициализира се при първото влизане в scope-а и се унищожава в края на програмата

- статична член-променлива - не е обвързана с конкретна инстанция на класа, а е обща за всички; глобален обект, капсулиран в дадения клас; не се ~~създава~~ създава или трие в класа и не влияе на размера му; инициализира се ~~вътрешен~~ (в .cpp файла на класа)

- статична член-функция - няма параметър this; не може да е const/virtual; има достъп само до статичните член-данни на класа

- статичен клас - има само статични член-данни (играе ролята на namespace)

• изключения – сигнал, че е възникнал проблем

throw <обект>

```
main{
  f();
}

f() {
  g();
}

g() {
  throw 1;
}
```

→ g() спира изпълнението си и поява stack unwinding

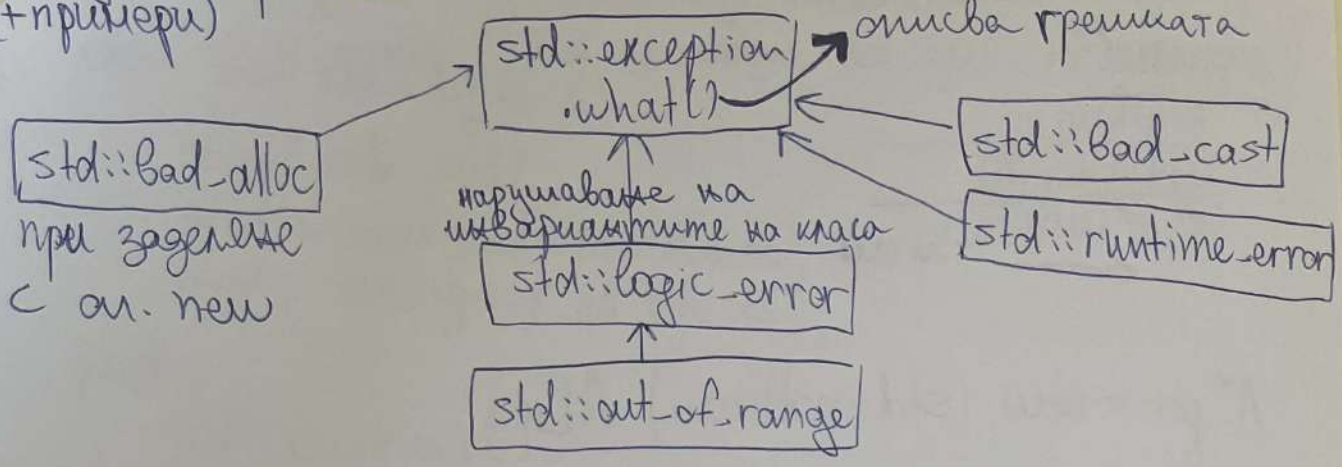
→ функцията, която е хвърлила грешка, прехвърля отговорността надолу по stack-а, докато не бъде обработена нейде

→ ако не бъде обработена → std::terminate
→ няма stack unwinding, т.е. не се викат деструк. на съществуващите обекти

→ обработка на грешки (тук се викат деструк. на обектите)

```
try { //проблемен код }
catch (<обект>) { //обработка }
catch(...) – хваща всякакъв exception
catch(const exception& e) → препоръчително
```

...дове грешки:
(+примери)




```
try {...}
catch (std::out_of_range) {...}
catch (std::logic_error) {...}
catch (std::bad_alloc) {...}
catch (std::exception) {...}
catch (...) {...}
```

хващане грешките от най-конкретна към най-обща

• изключения при деструктори

→ НЕ хвърляме, защото така може да има 2 грешки едновременно, което води до std::terminate

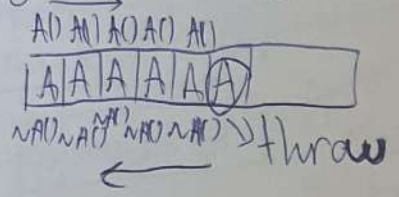
• изключения при конструктори

```
class A {
  B b;
  C c;
  D d;
  A() {
    throw 1;
  }
```

извикват се ~B(), ~C() и ~D(), понеже те са напълно построени, но ~A() не се вика, понеже той не е

→ НЕ трябва да оставаме отворени външни ресурси след хвърляне на изключение в конструктора

A* ptr = new A[10] → ако throw-не при ~~...~~ 6-тото заделяне на A, ще се извика ~A() 5 пъти



A* ptr = new (std::nothrow) A[10];

хваща грешката и ptr = nullptr;

• нива на exception safety

→ no throw guarantee (noexcept) – гарантираме, че няма ситуация, в която да хвърлим грешка

→ strong exception safety – гарантираме, че обектът няма да промени състоянието си, дори и при хвърляне на ~~грешка~~ изключение

→ ~~...~~ basic exception safety – възможно е да се хвърли изключение и да се промени състоянието на обекта, но въпреки това то ще е валидно

→ no exception safety – не гарантираме нищо